

Report file

THE ASSIGNMENT ENTERED IN THIS REPORT HAVE BEEN SATISFACTORILY
PERFORMED BY

Registration No: 25BAS10088

Name of Student: Anubhaw Anand Singh

Course Name: Fundamental in AI and ML

Course Code: CSA2001

School Name: SMEC

Class ID: BL2025260501076

Semester: Winter 2025/2026

Course Faculty Name: Dr. Mithun Singh Ahirwar

AI Gesture-Controlled System

(Volume, Brightness and Media Control using Hand Gestures)

1. Problem Statement

In our daily use of computers, we mostly depend on devices like keyboards, mice, or touchscreens to interact with the system. However, these methods are not always convenient or practical. For example, when our hands are occupied, dirty, or when we want a completely touch-free experience, these traditional input methods become limiting.

This project focuses on solving this problem by creating a system that allows users to control basic computer functions such as volume, brightness, and media playback using simple hand gestures captured through a webcam.

- **Core Problem:** How can we control system functions like volume, brightness, and media playback using natural hand gestures instead of traditional input devices?

2. Why This Problem Matters

This problem is important because it connects directly to how humans interact with technology.

Firstly, contactless interaction is becoming more relevant, especially in public or shared environments where hygiene is important. Secondly, gesture-based systems can improve accessibility for people who may have difficulty using traditional input devices.

Apart from that, this type of interaction is already being used in modern technologies like virtual reality, gaming, and smart devices. So, building such a system gives a practical understanding of future human-computer interaction.

- **Contactless Interaction**
 - Useful in public systems, hospitals, or shared devices
 - Reduces physical contact and improves hygiene
- **Accessibility**

- Helps users who have difficulty using keyboards or mice
- **Modern Technology Relevance**
 - Gesture control is widely used in:
 - AR/VR systems
 - Gaming
 - Smart TVs and devices
- **Convenience**
 - Faster and more intuitive way to control system functions

3. Approach & Methodology

Overall Approach

- Used a **rule-based computer vision system**
- No machine learning training required
- Real-time processing using webcam

Step 1: Hand Detection

I used the MediaPipe library to detect the hand from the webcam feed. It provides 21 landmark points on the hand, which include fingertips and joints.

Step 2: Landmark Processing

From these landmarks, I extracted the positions of important fingers such as the thumb, index, and middle finger. These points were then used to calculate distances and identify which fingers are up or down.

Step 3: Gesture Recognition

Defined simple rules instead of training a model:

- 🖐️ **Thumb + Index Distance**
 - Controls **volume**
- 🖐️ **Thumb + Middle Distance**
 - Controls **brightness**

- 🦊 **Closed Fist (All fingers down)**
 - Mute + Pause
- 🖐️ **Open Hand (All fingers up)**
 - Play
- 👉 **Only Index Finger Up**
 - Next Track
- 👍 **Only Thumb Up**
 - Previous Track

Step 4: System Control

Used different libraries for each function:

- 🔊 Volume → pycaw
- 💡 Brightness → screen-brightness-control
- 🎵 Media control → pyautogui

Step 5: Stability Improvement

- Implemented **gesture buffering**
 - Gesture must be stable for multiple frames
- Compared:
 - Current gesture vs previous gesture
- Result:
 - Reduced false detection
 - Prevented repeated triggering

4. Key Design Decisions

While building the project, I made some important decisions.

First, I chose a rule-based approach instead of machine learning because it is simpler, faster, and does not require training data. It also works well for basic gesture recognition.

Second, I used MediaPipe because it provides accurate and fast hand tracking, which is essential for real-time applications.

Another important decision was to use different gestures for different functions to avoid conflicts. For example, separate gestures were used for volume and brightness control.

Finally, I implemented gesture buffering to improve the overall stability and user experience.

- **✓ Rule-Based System**
 - Simple and fast
 - No dataset or training required
 - Suitable for real-time use
- **✓ Use of MediaPipe**
 - High accuracy
 - Lightweight and efficient
 - Easy to integrate
- **✓ Separate Gestures**
 - Avoids confusion between controls
 - Improves usability
- **✓ Gesture Buffering**
 - Ensures stable detection
 - Improves user experience

5. Challenges Faced

- **Library Compatibility Issues**
 - MediaPipe not working with Python 3.13
 - ✓ Solved by using Python 3.10
- **Continuous Triggering Problem**

- Play/Pause triggered repeatedly
- ✓ Fixed using gesture change detection
- **Gesture Misclassification**
 - Small hand movements caused wrong detection
 - ✓ Solved using gesture buffering
- **Installation Errors**
 - Issues with pycaw, pyautogui, etc.
 - ✓ Fixed by installing correct versions

6. Results & Output

- The system works in **real-time using webcam input**

✓ Achievements:

- Accurate hand detection
- Smooth gesture recognition
- Stable performance

✓ Functionalities:

- Volume control
- Brightness adjustment
- Media playback control
- Mute/unmute
- Track navigation

✓ UI Features:

- Gesture label display
- Volume & brightness bars
- FPS counter

7.What I Learned

This project helped me learn many practical concepts.

I gained a basic understanding of computer vision and how hand tracking works using MediaPipe. I also learned how to use mathematical concepts like distance calculation for real-time control.

Apart from technical knowledge, I learned how to debug real-world problems, handle library issues, and improve system performance.

Most importantly, I understood how simple logic can be used to build effective AI-based systems without always relying on complex models.

The important points from the learnings are:

- Basics of Computer Vision
- Working of MediaPipe hand tracking
- Use of distance calculation for control systems
- Importance of:
 - Logic design
 - Debugging
 - Stability improvements
- Learned how to:
 - Handle real-world coding issues
 - Work with multiple libraries together

8. Future Improvements

- Add *Machine Learning / Deep Learning model*
 - For more complex gesture recognition
- Implement:
 - Virtual mouse control
 - Gesture-based gaming
- Integrate with:
 - Smart home systems
 - IoT devices
- Add:

- GUI interface for better usability

9. Conclusion

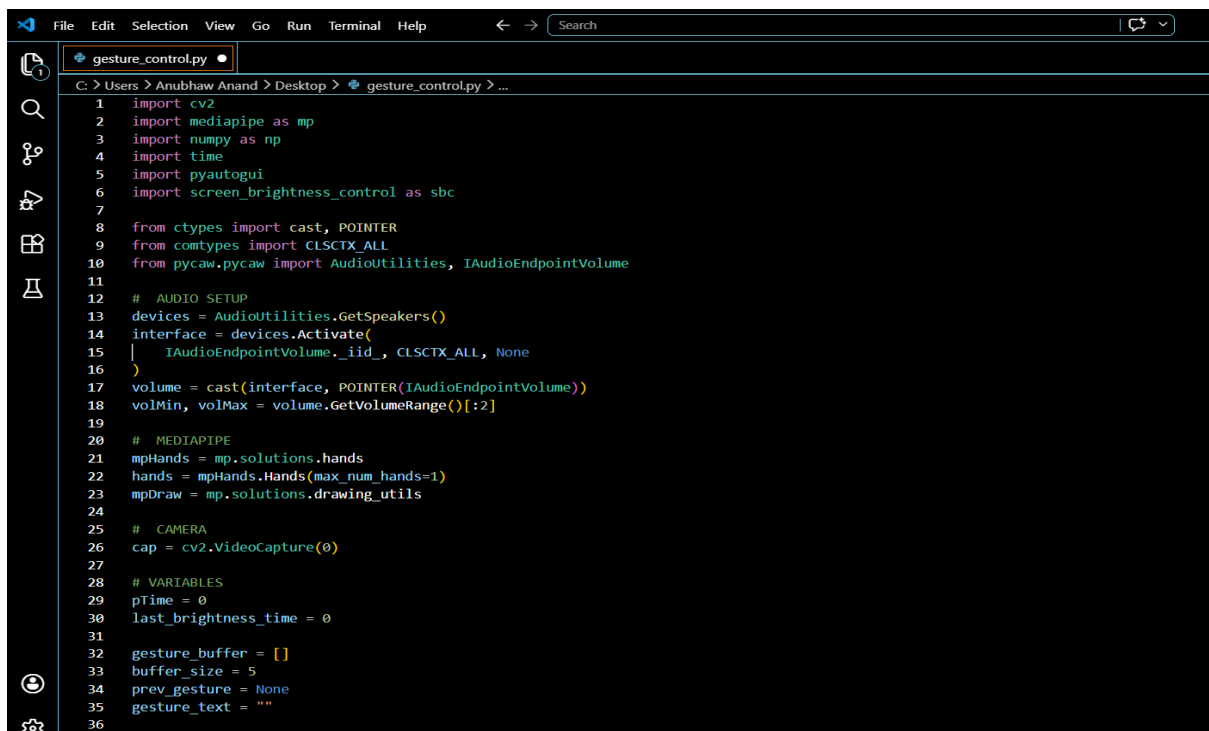
- This project demonstrates a simple yet effective gesture-controlled system using computer vision.
- Key highlights:
 - Real-time performance
 - No need for complex ML models
 - Practical and user-friendly
- It shows that:

Even basic logic and computer vision techniques can create powerful and interactive systems.

- The project provides a strong foundation for future work in:
 - Human-computer interaction
 - AI-based control systems

10. Snapshots

Input

A screenshot of a code editor window titled 'gesture_control.py'. The editor shows a Python script with line numbers 1 through 36. The script imports various libraries including cv2, mediapipe, numpy, time, pyautogui, and ctypes. It sets up audio and video capture, initializes Mediapipe hands detection, and defines variables for time, brightness, and gesture buffer. The code is as follows:

```
1 import cv2
2 import mediapipe as mp
3 import numpy as np
4 import time
5 import pyautogui
6 import screen_brightness_control as sbc
7
8 from ctypes import cast, POINTER
9 from comtypes import CLSCTX_ALL
10 from pycaw.pycaw import AudioUtilities, IAudioEndpointVolume
11
12 # AUDIO SETUP
13 devices = AudioUtilities.GetSpeakers()
14 interface = devices.Activate(
15     IAudioEndpointVolume._iid_, CLSCTX_ALL, None
16 )
17 volume = cast(interface, POINTER(IAudioEndpointVolume))
18 volMin, volMax = volume.GetVolumeRange()[2]
19
20 # MEDIAPIPE
21 mpHands = mp.solutions.hands
22 hands = mpHands.Hands(max_num_hands=1)
23 mpDraw = mp.solutions.drawing_utils
24
25 # CAMERA
26 cap = cv2.VideoCapture(0)
27
28 # VARIABLES
29 pTime = 0
30 last_brightness_time = 0
31
32 gesture_buffer = []
33 buffer_size = 5
34 prev_gesture = None
35 gesture_text = ""
36
```



```

36
37 # FUNCTIONS
38 def fingers_up(lmList):
39     fingers = []
40
41     # Thumb
42     if lmList[4][1] > lmList[3][1]:
43         fingers.append(1)
44     else:
45         fingers.append(0)
46
47     # Other fingers
48     tips = [8, 12, 16, 20]
49     for tip in tips:
50         if lmList[tip][2] < lmList[tip - 2][2]:
51             fingers.append(1)
52         else:
53             fingers.append(0)
54
55     return fingers
56
57
58 def get_gesture(fingers):
59     if fingers == [0,0,0,0,0]:
60         return "FIST"
61     elif fingers == [1,1,1,1,1]:
62         return "OPEN"
63     elif fingers == [0,1,0,0,0]:
64         return "NEXT"
65     elif fingers == [1,0,0,0,0]:
66         return "PREV"
67     else:
68         return "NONE"
69
70
71 # LOOP
72 while True:

```

```

68         return "NONE"
69
70
71 # LOOP
72 while True:
73     success, img = cap.read()
74     img = cv2.flip(img, 1)
75
76     imgRGB = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
77     results = hands.process(imgRGB)
78
79     lmList = []
80
81     if results.multi_hand_landmarks:
82         for handLms in results.multi_hand_landmarks:
83             for id, lm in enumerate(handLms.landmark):
84                 h, w, _ = img.shape
85                 cx, cy = int(lm.x * w), int(lm.y * h)
86                 lmList.append([id, cx, cy])
87
88             mpDraw.draw_landmarks(img, handLms, mpHands.HAND_CONNECTIONS)
89
90     if len(lmList) != 0:
91         fingers = fingers_up(lmList)
92
93         # VOLUME CONTROL
94         x1, y1 = lmList[4][1], lmList[4][2] # Thumb
95         x2, y2 = lmList[8][1], lmList[8][2] # Index
96
97         length = np.hypot(x2 - x1, y2 - y1)
98         vol = np.interp(length, [30, 200], [volMin, volMax])
99         volBar = np.interp(length, [30, 200], [400, 150])
100         volPer = np.interp(length, [30, 200], [0, 100])
101
102         volume.SetMasterVolumeLevel(vol, None)
103
104

```

```

102
103     volume.SetMasterVolumeLevel(vol, None)
104
105     cv2.rectangle(img, (50,150), (85,400), (0,255,0), 3)
106     cv2.rectangle(img, (50,int(volBar)), (85,400), (0,255,0), cv2.FILLED)
107     cv2.putText(img, "VOLUME", (40,140),
108     | | | cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0,255,0), 2)
109     cv2.putText(img, f'{int(volPer)}%', (40,450),
110     | | | cv2.FONT_HERSHEY_COMPLEX, 1, (0,255,0), 3)
111
112     # BRIGHTNESS CONTROL
113     x3, y3 = lmList[12][1], lmList[12][2] # Middle finger
114
115     length_b = np.hypot(x3 - x1, y3 - y1)
116     brightness = np.interp(length_b, [30, 200], [0, 100])
117     bar = np.interp(length_b, [30,200], [400,150])
118
119     if time.time() - last_brightness_time > 0.3:
120     |     sbc.set_brightness(int(brightness))
121     |     last_brightness_time = time.time()
122
123     cv2.rectangle(img, (150,150), (185,400), (255,0,0), 3)
124     cv2.rectangle(img, (150,int(bar)), (185,400), (255,0,0), cv2.FILLED)
125     cv2.putText(img, "BRIGHTNESS", (130,140),
126     | | | cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255,0,0), 2)
127     cv2.putText(img, f'{int(brightness)}%', (130,450),
128     | | | cv2.FONT_HERSHEY_COMPLEX, 1, (255,0,0), 3)
129
130     # GESTURE DETECTION
131     current_gesture = get_gesture(fingers)
132
133     gesture_buffer.append(current_gesture)
134     if len(gesture_buffer) > buffer_size:
135     |     gesture_buffer.pop(0)
136
137     if gesture_buffer.count(current_gesture) == buffer_size:

```

```

132
133     gesture_buffer.append(current_gesture)
134     if len(gesture_buffer) > buffer_size:
135     |     gesture_buffer.pop(0)
136
137     if gesture_buffer.count(current_gesture) == buffer_size:
138
139     |     if current_gesture != prev_gesture:
140     |
141     |         if current_gesture == "FIST":
142     |         |     pyautogui.press("volumemute")
143     |         |     pyautogui.press("playpause")
144     |         |     gesture_text = "MUTE / PAUSE"
145     |
146     |         elif current_gesture == "OPEN":
147     |         |     pyautogui.press("playpause")
148     |         |     gesture_text = "PLAY"
149     |
150     |         elif current_gesture == "NEXT":
151     |         |     pyautogui.press("nexttrack")
152     |         |     gesture_text = "NEXT TRACK"
153     |
154     |         elif current_gesture == "PREV":
155     |         |     pyautogui.press("prevtrack")
156     |         |     gesture_text = "PREVIOUS TRACK"
157     |
158     |         prev_gesture = current_gesture
159
160     # UI DISPLAY
161     cv2.rectangle(img, (350, 20), (640, 100), (0, 0, 0), -1)
162
163     cv2.putText(img, f'Gesture: {gesture_text}', (360, 70),
164     | | | cv2.FONT_HERSHEY_SIMPLEX, 0.9, (0,255,255), 2)
165
166     # | FPS
167     cTime = time.time()
168     fps = 1 / (cTime - pTime)

```

```

156         gesture_text = PREVIOUS_TRACK
157
158         prev_gesture = current_gesture
159
160     # UI DISPLAY
161     cv2.rectangle(img, (350, 20), (640, 100), (0, 0, 0), -1)
162
163     cv2.putText(img, f'Gesture: {gesture_text}', (360, 70),
164                 | cv2.FONT_HERSHEY_SIMPLEX, 0.9, (0,255,255), 2)
165
166     # FPS
167     cTime = time.time()
168     fps = 1 / (cTime - pTime)
169     pTime = cTime
170
171     cv2.putText(img, f'FPS: {int(fps)}', (500,450),
172                 | cv2.FONT_HERSHEY_COMPLEX, 1, (255,0,255), 2)
173
174     cv2.imshow("AI Gesture Control System", img)
175
176     if cv2.waitKey(1) & 0xFF == ord('q'):
177         break
178
179 cap.release()
180 cv2.destroyAllWindows()

```

Output

